Matplotlib Complete Guide

A-Level Computer Science

Drawing Grids and Paths for Pathfinding

A Comprehensive Reference with Examples

Contents

1 Introduction to Matplotlib

1.1 What is Matplotlib?

Matplotlib is Python's most popular library for creating visualisations. It can create:

- Line graphs, bar charts, scatter plots
- Heatmaps and images
- Custom shapes and drawings
- Interactive figures

For our pathfinding project, we use Matplotlib to:

1. Draw the grid map with coloured cells
2. Show walls, start, and end positions
3. Visualise the path our algorithm finds
4. Display which nodes were explored

1.2 Installation

Open your terminal or command prompt and run:

```
pip install matplotlib numpy
```

To verify the installation, open Python and type:

```
import matplotlib
print(matplotlib.__version__)
```

You should see a version number like `3.7.1` or similar.

1.3 The Two Main Modules We Use

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
```

Module	Purpose
`matplotlib.pyplot`	The main plotting interface. We import it as `plt` for convenience. Contains functions to create figures, add shapes, show plots, and save images.
`matplotlib.patches`	Contains shape classes like `Rectangle`, `Circle`, `Polygon`. We use `Rectangle` to draw grid cells.

2 Creating Figures and Axes

Every Matplotlib drawing needs two things:

1. A **Figure** – the window or canvas
2. An **Axes** – the drawing area inside the figure

2.1 The plt.subplots() Function

```
fig, ax = plt.subplots()
```

This creates both a figure and axes in one line. The function returns two objects:

- `fig` – the Figure object (the whole window)
- `ax` – the Axes object (where we draw)

2.1.1 Complete Parameter Reference for plt.subplots()

```
fig, ax = plt.subplots(
    nrows=1,            # Number of rows of subplots
    ncols=1,            # Number of columns of subplots
    figsize=(6.4, 4.8),# Figure size in inches (width, height)
    dpi=100,            # Dots per inch (resolution)
    facecolor='white', # Background colour of figure
    edgecolor='black', # Border colour of figure
    sharex=False,       # Share x-axis between subplots
    sharey=False        # Share y-axis between subplots
)
```

2.1.2 Parameter Details

Parameter	Type/Default	Description
`nrows`	int, default: 1	How many rows of subplots. Use 2 to have two plots stacked vertically.
`ncols`	int, default: 1	How many columns of subplots. Use 2 to have two plots side by side.
`figsize`	tuple (width, height), default: (6.4, 4.8)	Size of the figure in inches. For a square grid, use equal values like (8, 8).
`dpi`	int, default: 100	Resolution in dots per inch. Higher = sharper but larger file size.
`facecolor`	colour, default: 'white'	Background colour of the entire figure.
`edgecolor`	colour, default: 'black'	Border colour around the figure.

2.1.3 Examples with Different Sizes

```
# Small figure (good for thumbnails)
fig, ax = plt.subplots(figsize=(4, 4))

# Medium figure (good for most uses)
fig, ax = plt.subplots(figsize=(8, 8))
```

```
6
7  # Large figure (good for presentations)
8  fig, ax = plt.subplots(figsize=(12, 12))
9
10 # Match figure size to grid size
11 rows = 10
12 cols = 10
13 fig, ax = plt.subplots(figsize=(cols, rows))
```

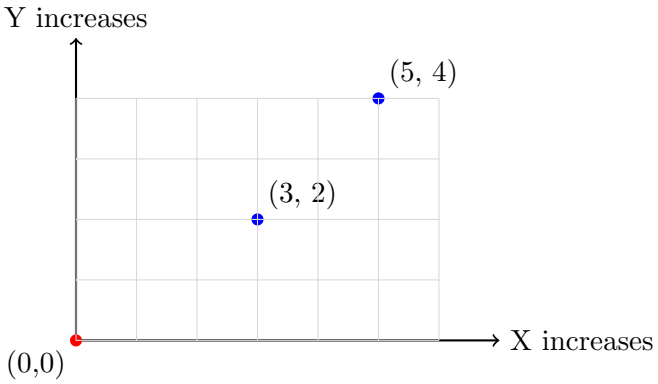
Tip: Matching Figure Size to Grid

For pathfinding grids, set `figsize=(cols, rows)` so each cell appears as a square. If your grid is 10x10, use `figsize=(10, 10)`.

3 Understanding the Coordinate System

This is **critically important** for drawing grids correctly!

3.1 Matplotlib's Coordinate System



- Origin **(0, 0)** is at the **bottom-left** corner
- **X** increases going **right**
- **Y** increases going **up**

3.2 Python Array Coordinate System

When we have a 2D array (our grid), the indexing works differently:

```
grid = [
    ['S', 0, 0, 0],     # Row 0 (TOP of visual grid)
    [0, 1, 1, 0],       # Row 1
    [0, 0, 0, 0],       # Row 2
    [0, 0, 0, 'E']      # Row 3 (BOTTOM of visual grid)
]

# Accessing elements:
grid[0][0]  # 'S' - top-left
grid[3][3]  # 'E' - bottom-right
grid[row][col]  # General access pattern
```

	Col 0	Col 1	Col 2	Col 3
Row 0	[0][0]	[0][1]	[0][2]	[0][3]
Row 1	[1][0]			
Row 2	[2][0]			
Row 3	[3][0]			[3][3]

- Row 0 is at the **top**
- Row increases going **down**
- Column increases going **right**

3.3 The Conversion Formula

To draw array position `(row, col)` in Matplotlib:

```
# Given:
row = 1       # Array row
col = 2       # Array column
total_rows = 4  # Total rows in grid

# Convert to Matplotlib coordinates:
matplotlib_x = col                      # X = column (same)
matplotlib_y = (total_rows - 1) - row   # Y = flipped row
```

Critical: Always Flip the Y Coordinate!

Without flipping: Row 0 appears at the bottom (wrong!)
With flipping: Row 0 appears at the top (correct!)
Formula: `matplotlib_y = (total_rows - 1) - row`

3.3.1 Conversion Examples

For a 5x5 grid (`total_rows = 5`):

Array (row, col)	Matplotlib (x, y)	Calculation
(0, 0)	(0, 4)	y = 5-1-0 = 4
(0, 4)	(4, 4)	y = 5-1-0 = 4
(2, 2)	(2, 2)	y = 5-1-2 = 2
(4, 0)	(0, 0)	y = 5-1-4 = 0
(4, 4)	(4, 0)	y = 5-1-4 = 0

4 Drawing Rectangles (Grid Cells)

We draw each cell as a `Rectangle` shape.

4.1 The plt.Rectangle() Function

```
rectangle = plt.Rectangle(
    xy,              # (x, y) position of bottom-left corner
    width,           # Width of rectangle
    height,          # Height of rectangle
    angle=0.0,       # Rotation angle in degrees
    facecolor=None,  # Fill colour (or 'fc')
    edgecolor=None,  # Border colour (or 'ec')
    linewidth=None,  # Border thickness (or 'lw')
    linestyle='-',   # Border style: '-', '--', ':', '-.'
    fill=True,       # Whether to fill with colour
    alpha=1.0        # Transparency (0=invisible, 1=solid)
)
```

4.1.1 Complete Parameter Reference

Parameter	Type/Default	Description
`xy`	tuple (x, y)	**Required.** The (x, y) coordinates of the bottom-left corner of the rectangle.
`width`	float	**Required.** Width of the rectangle. For grid cells, use 1.
`height`	float	**Required.** Height of the rectangle. For grid cells, use 1.
`angle`	float, default: 0	Rotation angle in degrees, counter-clockwise. Usually leave at 0.
`facecolor`	colour or None	Fill colour. Can use names ('red'), hex ('#FF0000'), or RGB tuples ((1,0,0)).
`edgecolor`	colour or None	Border colour. Set to 'black' for visible grid lines.
`linewidth`	float or None	Border thickness in points. Use 1-2 for normal grid lines.
`linestyle`	string, default: '-'	Border style: '-' solid, '–' dashed, ':' dotted, '-.' dash-dot.
`fill`	bool, default: True	If True, fill the rectangle with facecolor. If False, only draw border.
`alpha`	float, default: 1.0	Transparency. 0.0 = fully transparent, 1.0 = fully opaque, 0.5 = semi-transparent.

4.2 Adding Rectangles to the Axes

After creating a rectangle, you must add it to the axes:

```
# Create the rectangle
rect = plt.Rectangle((2, 3), 1, 1, facecolor='blue', edgecolor='black')

# Add it to the axes (REQUIRED - or it won't appear!)
ax.add_patch(rect)
```

Don't Forget add_patch()!

Creating a rectangle does NOT display it. You must call `ax.add_patch(rect)` to add it to your drawing.

4.3 Complete Example: Drawing a Single Cell

```
import matplotlib.pyplot as plt

# Step 1: Create figure and axes
fig, ax = plt.subplots(figsize=(6, 6))

# Step 2: Create a rectangle at position (2, 3)
# This is a 1x1 cell with red fill and black border
rect = plt.Rectangle(
    (2, 3),           # Bottom-left corner at x=2, y=3
    1,                # Width = 1
    1,                # Height = 1
    facecolor='red',  # Fill with red
    edgecolor='black',# Black border
    linewidth=2       # Border thickness = 2
)

# Step 3: Add rectangle to axes
ax.add_patch(rect)

# Step 4: Set axis limits so we can see the rectangle
ax.set_xlim(0, 6)
ax.set_ylim(0, 6)

# Step 5: Make sure cells look square
ax.set_aspect('equal')

# Step 6: Display
plt.show()
```

5 Specifying Colours

Matplotlib accepts colours in many formats.

5.1 Colour Format Options

Format	Example	Description
Named colour	`'red'`, `'blue'`, `'green'`	Simple colour names. Limited selection but easy to remember.
Single letter	`'r'`, `'b'`, `'g'`, `'k'`, `'w'`	Shortcuts: r=red, g=green, b=blue, c=cyan, m=magenta, y=yellow, k=black, w=white
Hex code	`'#FF0000'`, `'#4CAF50'`	6-digit hex RGB. Gives exact control over colour.
Hex with alpha	`'#FF0000AA'`	8-digit hex RGBA. Last 2 digits control transparency.
RGB tuple	`(1.0, 0.0, 0.0)`	RGB values from 0.0 to 1.0
RGBA tuple	`(1.0, 0.0, 0.0, 0.5)`	RGB + Alpha (transparency)
Grayscale	`'0.5'`	String number 0-1. '0'=black, '1'=white, '0.5'=gray

5.2 Colours for Pathfinding Grid

We use specific colours for each cell type:

Cell Type	Hex Code	RGB Tuple	Visual
Start (S)	`'#4CAF50'`	`(0.30, 0.69, 0.31)`	Green
End (E)	`'#F44336'`	`(0.96, 0.26, 0.21)`	Red
Open path (0)	`'#FFFFFF'`	`(1.0, 1.0, 1.0)`	White
Wall (1)	`'#424242'`	`(0.26, 0.26, 0.26)`	Dark Grey
Path found	`'#2196F3'`	`(0.13, 0.59, 0.95)`	Blue
Explored node	`'#FFEB3B'`	`(1.0, 0.92, 0.23)`	Yellow

5.3 Defining Colours as Constants

Good practice: define colours at the top of your file:

```
# Colour definitions for pathfinding grid
COLOUR_START = '#4CAF50'      # Green
COLOUR_END = '#F44336'        # Red
COLOUR_OPEN = '#FFFFFF'       # White
COLOUR_WALL = '#424242'       # Dark grey
COLOUR_PATH = '#2196F3'       # Blue
COLOUR_EXPLORED = '#FFEB3B'   # Yellow
COLOUR_BORDER = '#000000'     # Black

# Then use them:
rect = plt.Rectangle((0, 0), 1, 1, facecolor=COLOUR_START)
```

6 Configuring the Axes

After adding shapes, you need to configure the axes so everything displays correctly.

6.1 Setting Axis Limits

6.1.1 ax.set_xlim() and ax.set_ylim()

```
ax.set_xlim(left, right)  # Set x-axis range
ax.set_ylim(bottom, top)  # Set y-axis range
```

Parameters:

Parameter	Type	Description
`left`	float	Minimum x value (left edge of plot)
`right`	float	Maximum x value (right edge of plot)
`bottom`	float	Minimum y value (bottom edge of plot)
`top`	float	Maximum y value (top edge of plot)

For a grid with `rows` rows and `cols` columns:

```
ax.set_xlim(0, cols)  # x goes from 0 to number of columns
ax.set_ylim(0, rows)  # y goes from 0 to number of rows
```

6.2 Making Cells Square

6.2.1 ax.set_aspect()

```
ax.set_aspect(aspect, adjustable='box', anchor='C')
```

Parameters:

Parameter	Type	Description
`aspect`	'equal', 'auto', or float	'equal' makes 1 unit on x = 1 unit on y (squares look square). 'auto' lets axes scale independently. A number like 2.0 means y is 2x the scale of x.
`adjustable`	'box' or 'datalim'	How to achieve the aspect ratio. 'box' adjusts the axes box, 'datalim' adjusts the data limits.
`anchor`	string	Where to anchor the axes. 'C'=center, 'N'=north (top), etc.

For grids, always use:

```
ax.set_aspect('equal')
```

Without set_aspect('equal')

Your grid cells will appear stretched or squashed depending on the figure size. Always include this line!

6.3 Adding a Title

6.3.1 ax.set_title()

```
ax.set_title(
    label,              # The title text
    fontsize=12,        # Font size in points
    fontweight='normal',# 'normal', 'bold', 'light', etc.
    color='black',      # Text colour
```

```
    loc='center',        # Position: 'center', 'left', 'right'
    pad=6.0              # Padding between title and axes
)
```

Examples:

```
# Simple title
ax.set_title('My Grid')

# Formatted title
ax.set_title('Level 1 Solution', fontsize=16, fontweight='bold')

# Title with path information
path_length = len(path)
ax.set_title(f'Path Found: {path_length} steps', fontsize=14, color='green')
```

6.4 Hiding Axis Ticks and Labels

For a cleaner look, you can hide the axis numbers:

```
# Hide tick marks and labels
ax.set_xticks([])
ax.set_yticks([])

# Or hide the entire axis (including border)
ax.axis('off')
```

6.5 Adding Gridlines

```
ax.grid(
    visible=True,        # Show grid (True/False)
    which='major',       # 'major', 'minor', or 'both'
    axis='both',         # 'both', 'x', or 'y'
    color='gray',        # Grid line colour
    linestyle='-',       # Line style
    linewidth=0.5,       # Line thickness
    alpha=0.5            # Transparency
)
```

Note: For our grid cells, the rectangle borders act as gridlines, so we usually don't need `ax.grid()`.

7 Adding Text Labels

We add 'S' and 'E' labels to mark start and end cells.

7.1 The ax.text() Function

```
ax.text(
    x,                  # X position
    y,                  # Y position
    s,                  # The text string to display
    fontsize=12,        # Font size in points
    fontweight='normal',# 'normal', 'bold', 'light'
    color='black',      # Text colour
    ha='center',        # Horizontal alignment
    va='center',        # Vertical alignment
    fontfamily='sans-serif',  # Font family
    rotation=0,         # Rotation angle in degrees
    alpha=1.0           # Transparency
)
```

7.1.1 Complete Parameter Reference

Parameter	Type/Default	Description
`x`	float	X coordinate for the text position
`y`	float	Y coordinate for the text position
`s`	string	The text to display
`fontsize`	float, default: 12	Size of the font in points
`fontweight`	string, default: 'normal'	Weight of the font: 'ultralight', 'light', 'normal', 'regular', 'book', 'medium', 'roman', 'semibold', 'demibold', 'demi', 'bold', 'heavy', 'extra bold', 'black'
`color`	colour	Colour of the text
`ha`	string, default: 'left'	Horizontal alignment: 'left', 'center', 'right'
`va`	string, default: 'baseline'	Vertical alignment: 'top', 'center', 'bottom', 'baseline'
`rotation`	float, default: 0	Rotation angle in degrees
`alpha`	float, default: 1.0	Transparency (0-1)

7.2 Centering Text in a Cell

To center text in a grid cell at position (col, row):

```
# Cell is at matplotlib position (col, y) with size 1x1
# Center of cell is at (col + 0.5, y + 0.5)

col = 2
y = 3  # Remember: y is the flipped row

ax.text(
    col + 0.5,          # X: center of cell
    y + 0.5,            # Y: center of cell
    'S',                # Text to display
    ha='center',        # Horizontal: center
    va='center',        # Vertical: center
```

```
    fontsize=20,         # Large enough to see
    fontweight='bold',   # Bold for visibility
    color='white'        # White text on coloured background
)
```

7.3 Complete Example: Labels on Start and End

```
# Inside your grid-drawing loop:
for row in range(rows):
    for col in range(cols):
        cell = grid[row][col]
        y = rows - 1 - row  # Flip y coordinate

        # Draw the rectangle (code omitted for brevity)

        # Add labels for S and E
        if cell == 'S':
            ax.text(col + 0.5, y + 0.5, 'S',
                    ha='center', va='center',
                    fontsize=16, fontweight='bold',
                    color='white')
        elif cell == 'E':
            ax.text(col + 0.5, y + 0.5, 'E',
                    ha='center', va='center',
                    fontsize=16, fontweight='bold',
                    color='white')
```

8 Creating Legends

A legend explains what each colour means.

8.1 Using matplotlib.patches for Legend Items

```
import matplotlib.patches as mpatches

# Create a patch for the legend (not added to plot, just for legend)
patch = mpatches.Patch(
    facecolor='red',    # Fill colour
    edgecolor='black',  # Border colour
    label='My Label'    # Text shown in legend
)
```

8.2 The ax.legend() Function

```
ax.legend(
    handles=None,        # List of patches/artists to include
    labels=None,         # List of labels (if not using handle labels)
    loc='best',          # Location of legend
    fontsize='medium',   # Font size
    frameon=True,        # Draw frame around legend
    facecolor='white',   # Background colour
    edgecolor='black',   # Frame colour
    title=None,          # Legend title
    ncol=1,              # Number of columns
    bbox_to_anchor=None # Position anchor (for placing outside axes)
)
```

8.2.1 Location Options

Location String	Position
'best'	Automatic best location (avoids data)
'upper right'	Top-right corner
'upper left'	Top-left corner
'lower right'	Bottom-right corner
'lower left'	Bottom-left corner
'center'	Center of axes
'center right'	Center-right edge
'center left'	Center-left edge
'upper center'	Top-center
'lower center'	Bottom-center

8.3 Complete Legend Example

```
import matplotlib.patches as mpatches

# Create legend items
legend_items = [
    mpatches.Patch(facecolor='#4CAF50', edgecolor='black', label='Start'),
    mpatches.Patch(facecolor='#F44336', edgecolor='black', label='End'),
    mpatches.Patch(facecolor='#FFFFFF', edgecolor='black', label='Open'),
    mpatches.Patch(facecolor='#424242', edgecolor='black', label='Wall'),
    mpatches.Patch(facecolor='#2196F3', edgecolor='black', label='Path'),
    mpatches.Patch(facecolor='#FFEB3B', edgecolor='black', label='Explored')
]
```

```
12
13 # Add legend to axes
14 ax.legend(
15     handles=legend_items,
16     loc='upper left',
17     fontsize=10,
18     title='Legend'
19 )
```

8.4 Placing Legend Outside the Plot

If the legend overlaps your grid, place it outside:

```
1 ax.legend(
2     handles=legend_items,
3     loc='upper left',
4     bbox_to_anchor=(1.02, 1),  # Position to the right of axes
5     fontsize=10
6 )
7
8 # Adjust layout to make room for legend
9 plt.tight_layout()
```

9 Complete Grid Drawing Function

Now let's put everything together.

9.1 Function: Draw Basic Grid

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# Colour constants
COLOUR_START = '#4CAF50'
COLOUR_END = '#F44336'
COLOUR_OPEN = '#FFFFFF'
COLOUR_WALL = '#424242'
COLOUR_BORDER = '#000000'

def draw_grid(grid, title="Grid"):
    """
    Draw a grid map with coloured cells.

    Parameters:
    -----------
    grid : list of lists
        2D grid where:
        - 'S' = start position
        - 'E' = end position
        - 0 = open path
        - 1 = wall
    title : str
        Title to display above the grid
    """
    rows = len(grid)
    cols = len(grid[0])

    # Create figure and axes
    fig, ax = plt.subplots(figsize=(cols, rows))

    # Draw each cell
    for row in range(rows):
        for col in range(cols):
            cell = grid[row][col]

            # Determine colour based on cell type
            if cell == 'S':
                colour = COLOUR_START
            elif cell == 'E':
                colour = COLOUR_END
            elif cell == 1:
                colour = COLOUR_WALL
            else:
                colour = COLOUR_OPEN

            # Convert to matplotlib coordinates (flip y)
            y = rows - 1 - row

            # Create and add rectangle
            rect = plt.Rectangle(
                (col, y),           # Position
                1, 1,               # Size
                facecolor=colour,
                edgecolor=COLOUR_BORDER,
                linewidth=1
            )
            ax.add_patch(rect)

```

```
            # Add text labels for S and E
            if cell == 'S' or cell == 'E':
                ax.text(
                    col + 0.5, y + 0.5,
                    str(cell),
                    ha='center', va='center',
                    fontsize=14, fontweight='bold',
                    color='white'
                )

    # Configure axes
    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.set_aspect('equal')
    ax.set_title(title, fontsize=14, fontweight='bold')

    # Hide axis ticks
    ax.set_xticks([])
    ax.set_yticks([])

    plt.tight_layout()
    plt.show()
```

9.1.1 Usage Example

```
# Define a simple grid
my_grid = [
    ['S', 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 0, 0],
    [0, 0, 0, 1, 'E']
]

# Draw it
draw_grid(my_grid, "My Pathfinding Grid")
```

9.2 Function: Draw Grid with Path

```
COLOUR_PATH = '#2196F3'

def draw_grid_with_path(grid, path, title="Solution"):
    """
    Draw a grid map with the solution path highlighted.

    Parameters:
    -----------
    grid : list of lists
        2D grid map
    path : list of tuples
        List of (row, col) positions forming the path
        Example: [(0,0), (1,0), (2,0), (2,1)]
    title : str
        Title to display
    """
    rows = len(grid)
    cols = len(grid[0])

    fig, ax = plt.subplots(figsize=(cols, rows))

    for row in range(rows):
        for col in range(cols):
            cell = grid[row][col]

            # Determine colour - check path membership
            if cell == 'S':
                colour = COLOUR_START
            elif cell == 'E':
                colour = COLOUR_END
            elif (row, col) in path:
                colour = COLOUR_PATH  # Highlight path cells
            elif cell == 1:
                colour = COLOUR_WALL
            else:
                colour = COLOUR_OPEN

            y = rows - 1 - row

            rect = plt.Rectangle(
                (col, y), 1, 1,
                facecolor=colour,
                edgecolor=COLOUR_BORDER,
                linewidth=1
            )
            ax.add_patch(rect)

            if cell in ['S', 'E']:
                ax.text(
                    col + 0.5, y + 0.5, str(cell),
                    ha='center', va='center',
                    fontsize=14, fontweight='bold',
                    color='white'
                )

    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.set_aspect('equal')
    ax.set_title(f'{title} - Path Length: {len(path)}', fontsize=14)
    ax.set_xticks([])
    ax.set_yticks([])

    # Add legend
    legend_items = [
```

```
        mpatches.Patch(facecolor=COLOUR_START, edgecolor='black', label='Start')
    ,
        mpatches.Patch(facecolor=COLOUR_END, edgecolor='black', label='End'),
        mpatches.Patch(facecolor=COLOUR_PATH, edgecolor='black', label='Path'),
        mpatches.Patch(facecolor=COLOUR_WALL, edgecolor='black', label='Wall'),
    ]
    ax.legend(handles=legend_items, loc='upper left', bbox_to_anchor=(1.02, 1))

    plt.tight_layout()
    plt.show()
```

9.2.1 Usage Example

```
# The path your algorithm found
found_path = [(0,0), (1,0), (2,0), (2,1), (2,2), (3,2), (4,2), (4,3), (4,4)]

# Draw grid with path
draw_grid_with_path(my_grid, found_path, "Dijkstra Solution")
```

9.3 Function: Draw Grid with Path and Explored Nodes

```
COLOUR_EXPLORED = '#FFEB3B'

def draw_grid_with_explored(grid, path, explored, title="Solution"):
    """
    Draw grid showing both the path AND all explored nodes.

    Parameters:
    -----------
    grid : list of lists
        2D grid map
    path : list of tuples
        The solution path as (row, col) positions
    explored : set of tuples
        All nodes that were visited during search
    title : str
        Title to display
    """
    rows = len(grid)
    cols = len(grid[0])

    fig, ax = plt.subplots(figsize=(cols + 2, rows))  # Extra width for legend

    for row in range(rows):
        for col in range(cols):
            cell = grid[row][col]

            # Colour priority: Start/End > Path > Explored > Wall > Open
            if cell == 'S':
                colour = COLOUR_START
            elif cell == 'E':
                colour = COLOUR_END
            elif (row, col) in path:
                colour = COLOUR_PATH
            elif (row, col) in explored:
                colour = COLOUR_EXPLORED
            elif cell == 1:
                colour = COLOUR_WALL
            else:
                colour = COLOUR_OPEN

            y = rows - 1 - row

            rect = plt.Rectangle(
                (col, y), 1, 1,
                facecolor=colour,
                edgecolor=COLOUR_BORDER,
                linewidth=1
            )
            ax.add_patch(rect)

            if cell in ['S', 'E']:
                ax.text(
                    col + 0.5, y + 0.5, str(cell),
                    ha='center', va='center',
                    fontsize=14, fontweight='bold',
                    color='white'
                )

    ax.set_xlim(0, cols)
    ax.set_ylim(0, rows)
    ax.set_aspect('equal')
    ax.set_title(
        f'{title}\nPath: {len(path)} steps | Explored: {len(explored)} nodes',
        fontsize=12
```

```
    )
    ax.set_xticks([])
    ax.set_yticks([])

    # Complete legend
    legend_items = [
        mpatches.Patch(facecolor=COLOUR_START, edgecolor='black', label='Start')
    ,
        mpatches.Patch(facecolor=COLOUR_END, edgecolor='black', label='End'),
        mpatches.Patch(facecolor=COLOUR_PATH, edgecolor='black', label='Path'),
        mpatches.Patch(facecolor=COLOUR_EXPLORED, edgecolor='black', label='
    Explored'),
        mpatches.Patch(facecolor=COLOUR_WALL, edgecolor='black', label='Wall'),
        mpatches.Patch(facecolor=COLOUR_OPEN, edgecolor='black', label='Open'),
    ]
    ax.legend(handles=legend_items, loc='upper left', bbox_to_anchor=(1.02, 1))

    plt.tight_layout()
    plt.show()
```

Why Show Explored Nodes?

Showing explored nodes lets you compare algorithms:

- **Dijkstra** explores many nodes (spreads in all directions)
- **A*** explores fewer nodes (heads toward the goal)

Both find the same shortest path, but A* is more efficient!

10 Displaying and Saving Figures

10.1 Displaying the Figure

10.1.1 plt.show()

```
plt.show()
```

This opens a window displaying your figure. The program pauses until you close the window.
Parameters: None required for basic usage.

10.1.2 plt.tight_layout()

```
plt.tight_layout(
    pad=1.08,       # Padding around figure edges
    h_pad=None,     # Height padding between subplots
    w_pad=None,     # Width padding between subplots
    rect=None       # Rectangle for the tight layout (left, bottom, right, top)
)
```

Call this before `plt.show()` to automatically adjust spacing so nothing overlaps.

```
# Good practice:
plt.tight_layout()
plt.show()
```

10.2 Saving to a File

10.2.1 plt.savefig()

```
plt.savefig(
    fname,              # Filename (including extension)
    dpi=100,            # Resolution in dots per inch
    facecolor='white',  # Background colour
    edgecolor='white',  # Border colour
    format=None,        # File format (auto-detected from extension)
    bbox_inches=None,   # Bounding box: None, 'tight', or Bbox
    pad_inches=0.1,     # Padding when bbox_inches='tight'
    transparent=False   # Transparent background
)
```

10.2.2 Parameter Details

Parameter	Type/Default	Description
`fname`	string	Filename with extension. The extension determines the format: .png, .pdf, .svg, .jpg
`dpi`	int, default: 100	Resolution. Use 150-300 for print quality.
`bbox_inches`	'tight' or None	Use 'tight' to remove whitespace around the figure.
`transparent`	bool, default: False	If True, the background is transparent (useful for .png).

10.2.3 Examples

```python
# Save as PNG (good for web)
plt.savefig('my_grid.png', dpi=150, bbox_inches='tight')

# Save as high-resolution PNG (good for printing)
plt.savefig('my_grid_hires.png', dpi=300, bbox_inches='tight')

# Save as PDF (vector format, infinite resolution)
plt.savefig('my_grid.pdf', bbox_inches='tight')

# Save as SVG (vector format for web)
plt.savefig('my_grid.svg', bbox_inches='tight')

# Save with transparent background
plt.savefig('my_grid_transparent.png', transparent=True, bbox_inches='tight')
```

Order Matters!

Call `plt.savefig()` **before** `plt.show()`. After `show()`, the figure is cleared.

```python
# Correct order:
plt.tight_layout()
plt.savefig('output.png')  # Save first
plt.show()                 # Then display

# Wrong order (saves blank image):
plt.show()                 # Display first
plt.savefig('output.png')  # Figure already cleared!
```

11 Complete Reference Example

Here is a complete, well-commented example that you can copy and modify:

```
"""
Complete Matplotlib Grid Visualisation for Pathfinding
A-Level Computer Science

This file contains everything you need to visualise pathfinding grids.
"""

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# ============================================================================
# COLOUR DEFINITIONS
# ============================================================================
COLOUR_START = '#4CAF50'      # Green - start position
COLOUR_END = '#F44336'        # Red - end position
COLOUR_OPEN = '#FFFFFF'       # White - walkable cell
COLOUR_WALL = '#424242'       # Dark grey - blocked cell
COLOUR_PATH = '#2196F3'       # Blue - solution path
COLOUR_EXPLORED = '#FFEB3B'   # Yellow - explored nodes
COLOUR_BORDER = '#000000'     # Black - cell borders


# ============================================================================
# HELPER FUNCTION: Get colour for a cell
# ============================================================================
def get_cell_colour(cell, row, col, path, explored):
    """
    Determine what colour a cell should be.

    Priority order (highest to lowest):
    1. Start/End markers
    2. Path cells
    3. Explored cells
    4. Walls
    5. Open cells

    Parameters:
    -----------
    cell : str or int
        The cell value from the grid ('S', 'E', 0, or 1)
    row : int
        Row index of the cell
    col : int
        Column index of the cell
    path : list or None
        List of (row, col) tuples in the path
    explored : set or None
        Set of (row, col) tuples that were explored

    Returns:
    --------
    str : The hex colour code for this cell
    """
    if cell == 'S':
        return COLOUR_START
    elif cell == 'E':
        return COLOUR_END
    elif path is not None and (row, col) in path:
        return COLOUR_PATH
    elif explored is not None and (row, col) in explored:
        return COLOUR_EXPLORED
    elif cell == 1:
```

```
        return COLOUR_WALL
    else:
        return COLOUR_OPEN


# ============================================================================
# MAIN VISUALISATION FUNCTION
# ============================================================================
def visualise_grid(grid, path=None, explored=None, title="Grid",
                   show_legend=True, save_path=None):
    """
    Visualise a pathfinding grid with optional path and explored nodes.

    Parameters:
    -----------
    grid : list of lists
        2D grid where 'S'=start, 'E'=end, 0=open, 1=wall
    path : list of tuples, optional
        Solution path as [(row, col), ...]
    explored : set of tuples, optional
        Explored nodes as {(row, col), ...}
    title : str, optional
        Title to display above the grid
    show_legend : bool, optional
        Whether to show the colour legend
    save_path : str, optional
        If provided, save figure to this file path

    Returns:
    --------
    fig, ax : The matplotlib figure and axes objects

    Example:
    --------
    >>> grid = [['S', 0, 0], [1, 1, 0], [0, 0, 'E']]
    >>> path = [(0,0), (0,1), (0,2), (1,2), (2,2)]
    >>> visualise_grid(grid, path=path, title="My Solution")
    """
    # Get grid dimensions
    rows = len(grid)
    cols = len(grid[0])

    # Calculate figure size (add extra width if showing legend)
    fig_width = cols + (3 if show_legend else 0)
    fig_height = rows

    # Create figure and axes
    fig, ax = plt.subplots(figsize=(fig_width, fig_height))

    # ========================================================================
    # DRAW EACH CELL
    # ========================================================================
    for row in range(rows):
        for col in range(cols):
            # Get cell value
            cell = grid[row][col]

            # Get colour for this cell
            colour = get_cell_colour(cell, row, col, path, explored)

            # Convert array coordinates to matplotlib coordinates
            # Array row 0 is at TOP, matplotlib y=0 is at BOTTOM
            matplotlib_x = col
            matplotlib_y = (rows - 1) - row

            # Create rectangle for this cell
```

```
            rectangle = plt.Rectangle(
                xy=(matplotlib_x, matplotlib_y),  # Bottom-left corner
                width=1,
                height=1,
                facecolor=colour,
                edgecolor=COLOUR_BORDER,
                linewidth=1
            )

            # Add rectangle to axes
            ax.add_patch(rectangle)

            # Add text label for start and end
            if cell in ['S', 'E']:
                ax.text(
                    x=matplotlib_x + 0.5,      # Center of cell
                    y=matplotlib_y + 0.5,      # Center of cell
                    s=str(cell),               # The text
                    horizontalalignment='center',
                    verticalalignment='center',
                    fontsize=16,
                    fontweight='bold',
                    color='white'
                )

    # ================================================================
    # CONFIGURE AXES
    # ================================================================
    ax.set_xlim(0, cols)       # X range: 0 to number of columns
    ax.set_ylim(0, rows)       # Y range: 0 to number of rows
    ax.set_aspect('equal')     # Make cells square
    ax.set_xticks([])          # Hide x-axis ticks
    ax.set_yticks([])          # Hide y-axis ticks

    # ================================================================
    # ADD TITLE
    # ================================================================
    title_text = title
    if path is not None:
        title_text += f'\nPath Length: {len(path)} steps'
    if explored is not None:
        title_text += f' | Nodes Explored: {len(explored)}'

    ax.set_title(title_text, fontsize=14, fontweight='bold')

    # ================================================================
    # ADD LEGEND
    # ================================================================
    if show_legend:
        legend_items = [
            mpatches.Patch(facecolor=COLOUR_START,
                           edgecolor='black', label='Start (S)'),
            mpatches.Patch(facecolor=COLOUR_END,
                           edgecolor='black', label='End (E)'),
            mpatches.Patch(facecolor=COLOUR_OPEN,
                           edgecolor='black', label='Open'),
            mpatches.Patch(facecolor=COLOUR_WALL,
                           edgecolor='black', label='Wall'),
        ]

        # Add path and explored to legend only if they exist
        if path is not None:
            legend_items.append(
                mpatches.Patch(facecolor=COLOUR_PATH,
                               edgecolor='black', label='Path')
            )
```

```
        if explored is not None:
            legend_items.append(
                mpatches.Patch(facecolor=COLOUR_EXPLORED,
                               edgecolor='black', label='Explored')
            )

        ax.legend(
            handles=legend_items,
            loc='upper left',
            bbox_to_anchor=(1.02, 1),
            fontsize=10,
            title='Legend'
        )

    # ================================================================
    # FINALISE
    # ================================================================
    plt.tight_layout()

    # Save if path provided
    if save_path is not None:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        print(f"Figure saved to: {save_path}")

    # Display
    plt.show()

    return fig, ax


# ============================================================================
# EXAMPLE USAGE
# ============================================================================
if __name__ == "__main__":
    # Define a sample grid
    sample_grid = [
        ['S', 0,  0,  0,  0],
        [0,   1,  1,  1,  0],
        [0,   0,  0,  1,  0],
        [0,   1,  0,  0,  0],
        [0,   0,  0,  1, 'E']
    ]

    # Define a sample path
    sample_path = [
        (0, 0), (1, 0), (2, 0), (2, 1), (2, 2),
        (3, 2), (3, 3), (3, 4), (4, 4)
    ]

    # Define explored nodes
    sample_explored = {
        (0, 0), (0, 1), (1, 0), (2, 0), (2, 1), (2, 2),
        (3, 0), (3, 2), (3, 3), (3, 4), (4, 0), (4, 1),
        (4, 2), (4, 4)
    }

    # Example 1: Just the grid
    print("Example 1: Basic grid")
    visualise_grid(sample_grid, title="Basic Grid")

    # Example 2: Grid with path
    print("\nExample 2: Grid with path")
    visualise_grid(sample_grid, path=sample_path, title="Path Found")

    # Example 3: Grid with path and explored nodes
    print("\nExample 3: Grid with path and explored")
```

```
    visualise_grid(sample_grid, path=sample_path, explored=sample_explored,
                   title="Full Solution")

    # Example 4: Save to file
    print("\nExample 4: Saving to file")
    visualise_grid(sample_grid, path=sample_path, explored=sample_explored,
                   title="Saved Solution", save_path="solution.png")
```

12 Quick Reference Tables

12.1 Function Summary

Function	Purpose
`plt.subplots()`	Create a figure and axes
`plt.Rectangle()`	Create a rectangle shape
`ax.add_patch()`	Add a shape to the axes
`ax.text()`	Add text to the axes
`ax.set_xlim()`	Set x-axis range
`ax.set_ylim()`	Set y-axis range
`ax.set_aspect()`	Set aspect ratio ('equal' for squares)
`ax.set_title()`	Add a title
`ax.set_xticks()`	Set x-axis tick positions
`ax.set_yticks()`	Set y-axis tick positions
`ax.legend()`	Add a legend
`plt.tight_layout()`	Adjust spacing automatically
`plt.savefig()`	Save figure to a file
`plt.show()`	Display the figure

12.2 Colour Reference

Purpose	Hex Code	Named Colour
Start	`'#4CAF50'`	Material Green
End	`'#F44336'`	Material Red
Open	`'#FFFFFF'`	White
Wall	`'#424242'`	Dark Grey
Path	`'#2196F3'`	Material Blue
Explored	`'#FFEB3B'`	Material Yellow
Border	`'#000000'`	Black

12.3 Coordinate Conversion

Array Position	Matplotlib Position
Row	Y = (total_rows - 1) - row
Column	X = column (no change)
Cell center	(col + 0.5, y + 0.5)

12.4 Common Mistakes and Fixes

Problem	Solution
Rectangle not appearing	Add `ax.add_patch(rect)` after creating rectangle
Grid upside down	Use `y = (rows - 1) - row` conversion
Cells look rectangular	Add `ax.set_aspect('equal')`
Can't see all cells	Set correct limits: `ax.set_xlim(0, cols)`
Legend overlaps grid	Use `bbox_to_anchor=(1.02, 1)`
Saved file is blank	Call `savefig()` before `show()`
Labels cut off	Use `plt.tight_layout()`